

Módulo 3

Técnicas de Classificação

em Mineração de Dados

Conteúdo do Módulo:

- | | | | |
|--|--------------------------------------|--|---------------------------|
|  | Árvores de Decisão (ID3, C4.5, CART) |  | Redes Neurais Artificiais |
|  | Naive Bayes e Regressão Logística |  | SVM e Kernel Trick |
|  | Métodos Ensemble |  | Avaliação de Modelos |

Material didático para uso acadêmico. Baseado em Quinlan (1986, 1993), Breiman (2001), Vapnik (1995), Chen & Guestrin (2016) e outros.

Sumário

1	Objetivos de Aprendizagem	3
2	Introdução à Classificação	3
2.1	Aprendizado Supervisionado	3
2.2	Definição Formal do Problema de Classificação	4
2.3	Tipos de Classificação	4
2.4	Aplicações Reais	4
3	Árvores de Decisão	5
3.1	Conceito e Estrutura	5
3.2	Algoritmos ID3, C4.5 e CART	6
3.2.1	ID3 – Iterative Dichotomiser 3 (Quinlan, 1986)	6
3.2.2	C4.5 (Quinlan, 1993)	6
3.2.3	CART – Classification and Regression Trees (Breiman et al., 1984)	7
3.3	Poda (Pruning)	8
3.3.1	Pré-poda (Pre-pruning)	8
3.3.2	Pós-poda (Post-pruning)	8
4	Métodos Probabilísticos	8
4.1	Naive Bayes	8
4.1.1	Variantes do Naive Bayes	9
4.2	Regressão Logística	9
5	K-Vizinhos Mais Próximos (KNN)	10
5.1	Métricas de Distância	10
5.2	Efeito de k	10
5.3	Complexidade e Aceleração	10
6	Máquinas de Vetores de Suporte (SVM)	11
6.1	SVM Linear (Margem Rígida)	11
6.2	SVM com Margem Suave (Soft Margin)	11
6.3	Kernel Trick	12
7	Métodos Ensemble	12
7.1	Bagging e Random Forest	12
7.2	Boosting	13
7.2.1	AdaBoost (Freund & Schapire, 1997)	13
7.2.2	Gradient Boosting (Friedman, 2001)	13
7.2.3	XGBoost (Chen & Guestrin, 2016)	13
7.2.4	LightGBM (Ke et al., 2017)	14
7.2.5	CatBoost (Prokhorenkova et al., 2018)	14
7.3	Stacking	14

8	Avaliação de Modelos de Classificação	14
8.1	Métricas de Avaliação	14
8.1.1	Matriz de Confusão	14
8.1.2	Métricas Derivadas	15
8.2	Estratégias de Validação	16
8.3	Testes Estatísticos de Comparação	16
9	Estado da Arte (2020–2024)	17
10	Exercícios	18
11	Referências	19

1 Objetivos de Aprendizagem

Ao concluir este módulo, o estudante será capaz de:

- ✓ **Compreender** o problema de classificação supervisionada e sua formalização matemática, distinguindo classificação binária, multiclasse e multirótulo.
- ✓ **Construir** árvores de decisão manualmente utilizando os critérios de Entropia (ID3), Ganho de Informação com Razão de Ganho (C4.5) e Índice Gini (CART), identificando o melhor atributo de divisão em cada nível.
- ✓ **Aplicar** técnicas de poda (pre-pruning e post-pruning) para controlar o sobreajuste em árvores de decisão.
- ✓ **Implementar** classificadores probabilísticos (Naive Bayes Gaussiano, Multinomial, Bernoulli) e Regressão Logística, compreendendo as hipóteses de independência condicional e a função sigmoide.
- ✓ **Utilizar** Máquinas de Vetores de Suporte (SVM) com kernels linear, polinomial e RBF, entendendo o conceito de margem máxima, variáveis de folga e o kernel trick.
- ✓ **Construir** métodos ensemble (Bagging, Random Forest, AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost), comparando bagging versus boosting em termos de variância e viés.
- ✓ **Calcular e interpretar** métricas de avaliação de classificadores: acurácia, precisão, recall, F1-Score, AUC-ROC, AUC-PR, MCC e Kappa de Cohen, selecionando a métrica adequada conforme o contexto (dados desbalanceados, custo assimétrico de erros).
- ✓ **Interpretar** modelos de classificação usando técnicas de XAI (SHAP, LIME), justificando previsões individualmente e globalmente para suporte à tomada de decisão responsável.

2 Introdução à Classificação

2.1 Aprendizado Supervisionado

O **aprendizado supervisionado** é o paradigma de aprendizado de máquina no qual o algoritmo aprende uma função de mapeamento a partir de exemplos rotulados. O “supervisor” fornece pares entrada-saída $(\mathbf{x}_i, y_i)$ e o objetivo é generalizar para exemplos não vistos. Em contraste com o aprendizado não supervisionado (clusterização) e o aprendizado por reforço (recompensa), aqui cada exemplo de treino possui um rótulo de referência.

2.2 Definição Formal do Problema de Classificação

Problema de Classificação

Seja $\mathcal{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)\}$ um conjunto de treinamento, onde:

- $\mathbf{x}_i \in \mathbb{R}^p$ é o vetor de atributos (features) com p dimensões
- $y_i \in \{c_1, c_2, \dots, c_k\}$ é o rótulo de classe (k classes possíveis)

O objetivo é aprender uma função $f : \mathbb{R}^p \rightarrow \{c_1, \dots, c_k\}$ que minimize a perda esperada:

$$\mathcal{R}(f) = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{P}}[\mathcal{L}(y, f(\mathbf{x}))]$$

onde $\mathcal{P}$ é a distribuição de probabilidade desconhecida dos dados e $\mathcal{L}$ é a função de perda (e.g., 0-1 loss, log-loss).

Na prática, minimizamos o risco empírico (ERM – Empirical Risk Minimization):

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(y_i, f(\mathbf{x}_i)) + \lambda \Omega(f)$$

onde $\mathcal{F}$ é o espaço de hipóteses e $\Omega(f)$ é um termo de regularização.

2.3 Tipos de Classificação

Tipo	Definição	Exemplo
Binária	$k = 2: y \in \{0, 1\}$	Spam vs. Não-spam
Multiclasse	$k > 2: y \in \{c_1, \dots, c_k\}$	Dígito reconhecido (0–9)
Multirótulo	Múltiplos $y_j \in \{0, 1\}$ simultâneos	Gêneros de um filme
Multiclasse-multirótulo	Combinação	Análise de emoções em texto

2.4 Aplicações Reais

-  **Filtragem de spam:** classificar e-mails como spam/não-spam com base em palavras e metadados.
-  **Diagnóstico médico:** identificar tumor maligno/benigno a partir de imagens histológicas ou dados clínicos.
-  **Risco de crédito:** aprovar ou negar empréstimos com base em histórico financeiro do cliente.
-  **Reconhecimento de imagens:** identificar objetos, faces, expressões em fotografias (ImageNet, CIFAR-10).
-  **Churn prediction:** prever quais clientes irão cancelar assinatura para retenção proativa.

- **🛡️ Detecção de fraudes:** identificar transações financeiras fraudulentas em tempo real.
- **🗣️ Análise de sentimentos:** classificar opiniões de usuários em positivo/negativo/neutro.
- **🧬 Bioinformática:** classificar sequências genéticas ou prever função de proteínas.

3 Árvores de Decisão

3.1 Conceito e Estrutura

Uma **árvore de decisão** é um modelo hierárquico que representa uma série de decisões sequenciais na forma de uma estrutura de árvore. Cada *nó interno* contém um teste sobre um atributo; cada *ramo* representa um resultado do teste; cada *folha* atribui um rótulo de classe. Para classificar um novo exemplo, percorremos a árvore da raiz até uma folha, seguindo os ramos correspondentes aos valores dos atributos do exemplo.

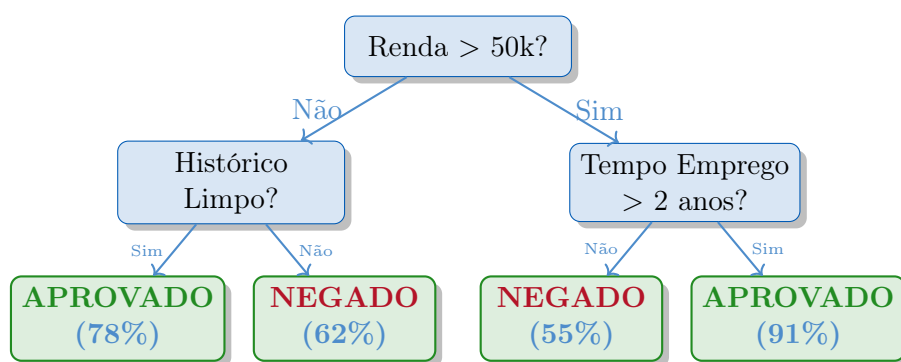


Figura 1: Exemplo de árvore de decisão para aprovação de crédito. Nós internos: condições de teste. Folhas: classes com confiança estimada.

i Propriedades das Árvores de Decisão

- **Interpretabilidade:** regras explícitas, legíveis por humanos (“Se renda > 50k E emprego > 2 anos, ENTÃO aprovado”).
- **Não-parametricidade:** não assume distribuição dos dados.
- **Mistura de tipos:** lidam nativamente com atributos numéricos e categóricos.
- **Ausência de normalização:** não requer escalonamento de features.
- **Instabilidade:** pequenas mudanças nos dados podem gerar árvores muito diferentes (mitigado por ensembles).

3.2 Algoritmos ID3, C4.5 e CART

3.2.1 ID3 – Iterative Dichotomiser 3 (Quinlan, 1986)

O ID3 seleciona o atributo que maximiza o **Ganho de Informação**. O critério de impureza utilizado é a **Entropia de Shannon**:

$$H(S) = - \sum_{i=1}^k p_i \log_2 p_i$$

onde $p_i = \frac{|S_i|}{|S|}$ é a proporção de exemplos da classe c_i no conjunto S . A entropia é máxima ($H = \log_2 k$) quando as classes são equiprováveis e zero quando o conjunto é puro.

O **Ganho de Informação** de um atributo A com domínio $V(A)$:

$$IG(S, A) = H(S) - \sum_{v \in V(A)} \frac{|S_v|}{|S|} H(S_v)$$

💡 Exemplo Calculado – Ganho de Informação

Considere um dataset de 14 exemplos (9 positivos, 5 negativos) com atributo *Vento* (Fraco/Forte):

- $H(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} \approx 0,940$ bits
- $S_{\text{Fraco}} = \{6^+, 2^-\}$: $H = -\frac{6}{8} \log_2 \frac{6}{8} - \frac{2}{8} \log_2 \frac{2}{8} \approx 0,811$
- $S_{\text{Forte}} = \{3^+, 3^-\}$: $H = 1,000$ (equiprável)
- $IG(\text{Vento}) = 0,940 - (\frac{8}{14} \cdot 0,811 + \frac{6}{14} \cdot 1,000) \approx 0,048$ bits

O atributo com maior IG é escolhido como nó raiz.

Limitações do ID3: favorece atributos com muitos valores distintos (e.g., ID do cliente tem IG alto mas é inútil para generalização).

3.2.2 C4.5 (Quinlan, 1993)

O C4.5 corrige o viés do ID3 usando a **Razão de Ganho** (*Gain Ratio*):

$$GR(S, A) = \frac{IG(S, A)}{H_A(S)}$$

onde $H_A(S)$ é a **entropia de divisão** do atributo A :

$$H_A(S) = - \sum_{v \in V(A)} \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}$$

Atributos com muitos valores terão alto $H_A(S)$, penalizando seu ganho ratio. O C4.5 também suporta: (1) atributos contínuos (via thresholds), (2) valores ausentes, (3) poda baseada em confiança estatística.

3.2.3 CART – Classification and Regression Trees (Breiman et al., 1984)

O CART usa o **Índice Gini** como critério de impureza:

$$Gini(S) = 1 - \sum_{i=1}^k p_i^2$$

O Gini mede a probabilidade de classificar incorretamente um exemplo escolhido aleatoriamente. Para dividir o conjunto S pelo atributo A :

$$GiniGain(S, A) = Gini(S) - \sum_{v \in V(A)} \frac{|S_v|}{|S|} Gini(S_v)$$

⚠ Diferenças práticas entre ID3, C4.5 e CART

Característica	ID3	C4.5	CART
Critério	Entropia/IG	Gain Ratio	Gini
Divisão	Multiway	Multiway	Binária
Valores contínuos	Não	Sim	Sim
Valores ausentes	Não	Sim	Sim (surrogate)
Regressão	Não	Não	Sim
Poda	Não	Pós-poda (CF)	Custo-complexidade

O pseudocódigo do ID3 recursivo:

Algorithm 1: Algoritmo ID3 Recursivo

Input: Conjunto S , conjunto de atributos $\mathcal{A}$
Output: Árvore de Decisão T

```

1 if  $S$  é puro (todos os exemplos têm a mesma classe  $c$ ) then
2   | return folha com rótulo  $c$ ;
3 end
4 if  $\mathcal{A} = \emptyset$  ou critério de parada satisfeito then
5   | return folha com classe majoritária em  $S$ ;
6 end
7  $A^* \leftarrow \arg \max_{A \in \mathcal{A}} IG(S, A)$ ;
8 Cria nó interno  $N$  com teste no atributo  $A^*$ ;
9 foreach valor  $v \in V(A^*)$  do
10  |  $S_v \leftarrow \{\mathbf{x} \in S : x_{A^*} = v\}$ ;
11  | if  $S_v = \emptyset$  then
12  |   | Adiciona folha com classe majoritária de  $S$  ao nó  $N$ ;
13  | else
14  |   | Adiciona ramo  $v$  com subárvore  $ID3(S_v, \mathcal{A} \setminus \{A^*\})$  ao nó  $N$ ;
15  | end
16 end
17 return nó  $N$ ;

```

3.3 Poda (Pruning)

Árvores completamente expandidas tendem a sobreajustar (overfitting) os dados de treinamento. A **poda** reduz a complexidade da árvore para melhorar a generalização.

3.3.1 Pré-poda (Pre-pruning)

Critérios de parada antecipada durante a construção:

- **max_depth**: profundidade máxima permitida
- **min_samples_split**: mínimo de exemplos para dividir um nó
- **min_samples_leaf**: mínimo de exemplos em uma folha
- **max_leaf_nodes**: número máximo de folhas
- Teste chi-quadrado ou teste-G para avaliar significância da divisão

3.3.2 Pós-poda (Post-pruning)

Cost-Complexity Pruning (CART): define o critério de custo-complexidade:

$$R_\alpha(T) = R(T) + \alpha \cdot |T|$$

onde $R(T)$ é o erro de classificação, $|T|$ é o número de folhas e $\alpha \geq 0$ é o parâmetro de regularização. Para cada α , existe a subárvore ótima $T(\alpha)$.

Reduced Error Pruning (C4.5): usando um conjunto de validação, remove subárvores cujos nós raiz (convertidos em folhas) não aumentam o erro no conjunto de validação.

4 Métodos Probabilísticos

4.1 Naive Bayes

O classificador **Naive Bayes** aplica o Teorema de Bayes com a hipótese (ingênua) de independência condicional entre os atributos dada a classe.

Teorema de Bayes:

$$P(c \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid c) P(c)}{P(\mathbf{x})}$$

Hipótese de Naive Bayes: os atributos são condicionalmente independentes dada a classe:

$$P(\mathbf{x} \mid c) = \prod_{j=1}^p P(x_j \mid c)$$

A regra de classificação (maximizar a probabilidade a posteriori):

$$\hat{y} = \arg \max_{c \in \mathcal{C}} P(c) \prod_{j=1}^p P(x_j \mid c)$$

4.1.1 Variantes do Naive Bayes

Variante	Distribuição assumida	Aplicação típica
Gaussian NB	$P(x_j c) = \mathcal{N}(\mu_{jc}, \sigma_{jc}^2)$	Features contínuas
Multinomial NB	Distribuição multinomial	Classificação de textos (freq. palavras)
Bernoulli NB	Distribuição de Bernoulli	Features binárias (presença/ausência)
Complement NB	Modela complemento das classes	Datasets desbalanceados

Suavização de Laplace: evita probabilidades zero quando um atributo não aparece com determinada classe no treino:

$$P(x_j = v \mid c) = \frac{\text{count}(x_j = v, c) + \alpha}{\text{count}(c) + \alpha \cdot |V_j|}$$

onde $\alpha = 1$ (suavização de Laplace) ou $0 < \alpha < 1$ (Lidstone smoothing), e $|V_j|$ é o número de valores distintos do atributo j .

4.2 Regressão Logística

A **Regressão Logística** é um classificador linear que modela a probabilidade da classe positiva usando a função sigmoide:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = \mathbf{w}^T \mathbf{x} + b$$

A saída $P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b)$ é otimizada minimizando a **cross-entropy loss**:

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

Os parâmetros são estimados por gradiente descendente (ou LBFGS, Newton-CG):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) \mathbf{x}_i$$

Softmax para multiclasse: generaliza a sigmoide para k classes:

$$P(y = k \mid \mathbf{x}) = \frac{e^{\mathbf{w}_k^T \mathbf{x}}}{\sum_{j=1}^K e^{\mathbf{w}_j^T \mathbf{x}}}$$

i Regularização na Regressão Logística

Para evitar sobreajuste, adiciona-se penalidade: $\mathcal{L}_{reg} = \mathcal{L} + \frac{\lambda}{2} \|\mathbf{w}\|^2$ (L2/Ridge) ou $\mathcal{L}_{reg} = \mathcal{L} + \lambda \|\mathbf{w}\|_1$ (L1/LASSO). O parâmetro $C = 1/\lambda$ no scikit-learn controla a regularização.

5 K-Vizinhos Mais Próximos (KNN)

O algoritmo **K-Vizinhos Mais Próximos (KNN)** é um classificador não-paramétrico baseado em instâncias (lazy learning). Para classificar $\mathbf{x}$: encontre os k exemplos de treino mais próximos e classifique por voto majoritário.

5.1 Métricas de Distância

Métrica	Fórmula	Uso
Euclidiana	$d(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_i (a_i - b_i)^2}$	Dados contínuos
Manhattan	$d(\mathbf{a}, \mathbf{b}) = \sum_i a_i - b_i $	Menos sensível a outliers
Minkowski	$d(\mathbf{a}, \mathbf{b}) = (\sum_i a_i - b_i ^r)^{1/r}$	Generalização ($r = 2$: Euclid.)
Hamming	$d(\mathbf{a}, \mathbf{b}) = \frac{1}{p} \sum_i \mathbf{1}[a_i \neq b_i]$	Atributos categóricos
Cosine	$d(\mathbf{a}, \mathbf{b}) = 1 - \frac{\mathbf{a} \cdot \mathbf{b}}{\ \mathbf{a}\ \ \mathbf{b}\ }$	Textos/vetores de palavras

5.2 Efeito de k

- k pequeno (e.g., $k = 1$): alta variância, baixo viés – fronteira de decisão irregular, sensível a ruído (**overfitting**)
- k grande (e.g., $k = n$): baixa variância, alto viés – fronteira suave, mas ignora estrutura local (**underfitting**)
- Escolha de k via validação cruzada; tipicamente $k = \sqrt{n}$ como ponto de partida

KNN Ponderado: pondera cada vizinho pelo inverso da distância ao quadrado:

$$\hat{y} = \frac{\sum_{i \in N_k(\mathbf{x})} w_i y_i}{\sum_{i \in N_k(\mathbf{x})} w_i}, \quad w_i = \frac{1}{d(\mathbf{x}, \mathbf{x}_i)^2 + \varepsilon}$$

5.3 Complexidade e Aceleração

- **Predição ingênua:** $O(n \cdot p)$ por consulta – inviável para grandes datasets
- **KD-tree:** particiona o espaço de features em hiperretângulos; $O(p \log n)$ para dimensões baixas ($p \lesssim 20$)
- **Ball-tree:** usa hipersféricas; robusto para dimensões maiores e métricas não-Euclidianas
- **Approximate Nearest Neighbors (ANN):** FAISS, HNSW para alta dimensão (e.g., embeddings)

⚠ Maldição da Dimensionalidade

Em altas dimensões ($p \gg 1$), todos os pontos tendem a estar aproximadamente equidistantes. KNN sofre severamente neste regime. Pré-processamento com PCA ou outros métodos de redução de dimensionalidade pode mitigar o problema.

6 Máquinas de Vetores de Suporte (SVM)

6.1 SVM Linear (Margem Rígida)

Uma **Máquina de Vetores de Suporte** encontra o hiperplano de separação que maximiza a **margem** entre as classes. O hiperplano é definido por:

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

A distância de um ponto ao hiperplano é $\frac{|\mathbf{w} \cdot \mathbf{x} + b|}{\|\mathbf{w}\|}$. A margem entre as classes é $\frac{2}{\|\mathbf{w}\|}$. Para classes linearmente separáveis, o problema de otimização (margem rígida):

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.a.} \quad y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1, \quad \forall i$$

Os exemplos que satisfazem $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) = 1$ são os **vetores de suporte**.

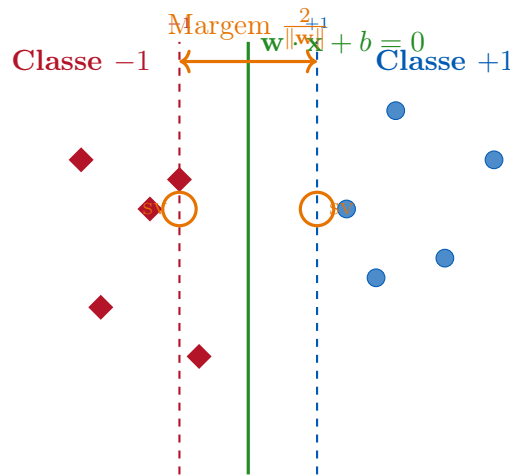


Figura 2: SVM com margem máxima. Os vetores de suporte (SVs) são os exemplos mais próximos do hiperplano e determinam a fronteira de decisão.

6.2 SVM com Margem Suave (Soft Margin)

Para dados não linearmente separáveis, introduzem-se **variáveis de folga** $\xi_i \geq 0$:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.a.} \quad y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

O parâmetro $C > 0$ controla o trade-off entre maximizar a margem e minimizar as violações:

- C grande: penaliza fortemente erros – margem menor, menos erros de treino (risco de overfitting)
- C pequeno: aceita mais erros – margem maior, melhor generalização

6.3 Kernel Trick

O **kernel trick** permite que a SVM opere em espaços de alta dimensão sem calcular explicitamente o mapeamento $\phi : \mathbb{R}^p \rightarrow \mathcal{H}$, usando apenas o produto interno $K(\mathbf{x}, \mathbf{z}) = \langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle$.

Kernel	Fórmula	Uso
Linear	$K(\mathbf{x}, \mathbf{z}) = \mathbf{x} \cdot \mathbf{z}$	Dados linearmente separáveis
Polinomial	$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x} \cdot \mathbf{z} + c)^d$	Interações de grau d
RBF (Gaussiano)	$K(\mathbf{x}, \mathbf{z}) = e^{-\gamma \ \mathbf{x} - \mathbf{z}\ ^2}$	Fronteiras complexas (padrão)
Sigmoide	$K(\mathbf{x}, \mathbf{z}) = \tanh(\kappa \mathbf{x} \cdot \mathbf{z} + c)$	Inspirado em redes neurais

Teorema de Mercer

Uma função $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ é um kernel válido (positivo semi-definido) se e somente se, para qualquer conjunto finito $\{x_1, \dots, x_n\}$, a matriz de Gram $G_{ij} = K(x_i, x_j)$ é positiva semi-definida. Kernels válidos garantem que o produto interno no espaço de features seja bem-definido.

7 Métodos Ensemble

Métodos ensemble combinam múltiplos classificadores para obter desempenho superior ao de qualquer modelo individual. Baseiam-se na **decomposição viés-variância**: $\text{Erro} = \text{Viés}^2 + \text{Variância} + \text{Ruído irreduzível}$.

7.1 Bagging e Random Forest

Bagging (Bootstrap AGGREGatING, Breiman 1996): treina T modelos em subamostras bootstrap (amostras com reposição de tamanho n do dataset original) e combina por votação majoritária (classificação) ou média (regressão). Reduz principalmente a **variância**.

Random Forest (Breiman, 2001)

Random Forest é um ensemble de T árvores de decisão onde:

1. Cada árvore é treinada em uma amostra bootstrap distinta do dataset de treino
2. Em cada divisão de nó, apenas um subconjunto aleatório de $m \approx \sqrt{p}$ features (classificação) ou $m \approx p/3$ (regressão) é avaliado

3. A predição final é a votação majoritária das T árvores

A seleção aleatória de features reduz a correlação entre as árvores, diminuindo a variância do ensemble além do que seria possível com bagging puro.

Medidas de importância de features em Random Forest:

- **MDI (Mean Decrease in Impurity):** média da redução de Gini/Entropia ponderada pelos exemplos, somada sobre todas as árvores. Rápida mas enviesada para features de alta cardinalidade.
- **MDA (Mean Decrease in Accuracy):** permuta aleatoriamente os valores de cada feature e mede queda na acurácia OOB. Mais confiável, mais custosa.
- **SHAP:** atribuição baseada em valores de Shapley, teoricamente fundamentada.

7.2 Boosting

Boosting treina modelos sequencialmente, onde cada modelo subsequente foca nos erros do modelo anterior. Reduz principalmente o **viés**.

7.2.1 AdaBoost (Freund & Schapire, 1997)

1. Inicializa pesos uniformes $w_i = 1/n$
2. Para $m = 1, \dots, M$: treina classificador h_m nos pesos atuais; calcula erro $\varepsilon_m = \sum_i w_i \mathbf{1}[h_m(\mathbf{x}_i) \neq y_i]$; calcula $\alpha_m = \frac{1}{2} \ln \frac{1-\varepsilon_m}{\varepsilon_m}$; atualiza pesos e normaliza
3. Predição: $H(\mathbf{x}) = \text{sign}(\sum_m \alpha_m h_m(\mathbf{x}))$

7.2.2 Gradient Boosting (Friedman, 2001)

Generalização que enquadra boosting como otimização em espaço funcional:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \gamma_m h_m(\mathbf{x})$$

onde h_m é ajustado aos **pseudo-resíduos** $r_{im} = - \left[\frac{\partial \mathcal{L}(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right]_{F=F_{m-1}}$.

7.2.3 XGBoost (Chen & Guestrin, 2016)

Objetivo regularizado de segunda ordem:

$$\mathcal{L}(\phi) = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

onde T é o número de folhas e w são os pesos das folhas. Usa expansão de Taylor de segunda ordem para otimização exata.

7.2.4 LightGBM (Ke et al., 2017)

Inovações-chave: (1) **GOSS** (Gradient-based One-Side Sampling): mantém exemplos com gradientes grandes, amostra aleatoriamente os de gradiente pequeno; (2) **EFB** (Exclusive Feature Bundling): agrupa features mutuamente exclusivas para reduzir dimensionalidade; (3) Crescimento **leaf-wise** (vs. level-wise do XGBoost): divide a folha com maior ganho.

7.2.5 CatBoost (Prokhorenkova et al., 2018)

Ordered boosting: usa permutações aleatórias dos dados de treino para calcular estatísticas de target encoding sem data leakage. Suporte nativo a features categóricas de alta cardinalidade.

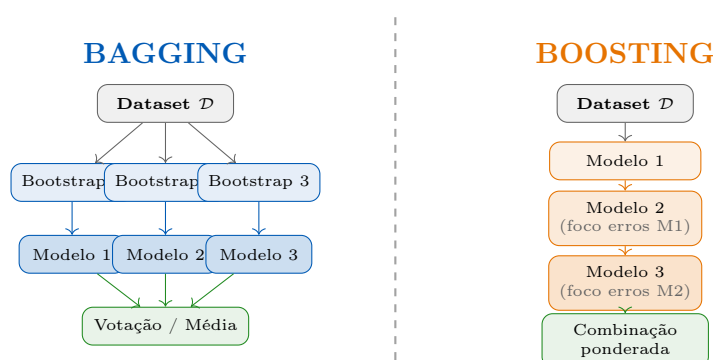


Figura 3: Comparação entre Bagging (paralelo, independente) e Boosting (sequencial, adaptativo).

7.3 Stacking

O **Stacking** (Wolpert, 1992) usa as previsões de modelos base (*nível 0*) como features de entrada para um meta-modelo (*nível 1*):

1. Treina L modelos base $\{h_1, \dots, h_L\}$ via cross-validation (para evitar data leakage)
2. Para cada exemplo, coleta as previsões dos L modelos como novo vetor de features
3. Treina um meta-modelo (frequentemente regressão logística ou XGBoost) nas previsões dos modelos base

8 Avaliação de Modelos de Classificação

8.1 Métricas de Avaliação

8.1.1 Matriz de Confusão

Para classificação binária, a matriz de confusão organiza as previsões:

	Previsto +	Previsto -
Real +	VP (Verdadeiro Positivo)	FN (Falso Negativo)
Real -	FP (Falso Positivo)	VN (Verdadeiro Negativo)

Figura 4: Matriz de confusão para classificação binária. Diagonal principal (VP, VN): predições corretas. Fora da diagonal (FN, FP): erros de classificação.

8.1.2 Métricas Derivadas

$$\text{Acurácia} = \frac{VP + VN}{VP + VN + FP + FN} \quad (1)$$

$$\text{Precisão} = \frac{VP}{VP + FP} \quad (\text{fração dos positivos preditos que são corretos}) \quad (2)$$

$$\text{Recall (Sensibilidade)} = \frac{VP}{VP + FN} \quad (\text{fração dos positivos reais detectados}) \quad (3)$$

$$\text{Especificidade} = \frac{VN}{VN + FP} \quad (4)$$

$$\text{F1-Score} = 2 \cdot \frac{\text{Precisão} \cdot \text{Recall}}{\text{Precisão} + \text{Recall}} \quad (5)$$

$$F_\beta = \frac{(1 + \beta^2) \cdot \text{Precisão} \cdot \text{Recall}}{\beta^2 \cdot \text{Precisão} + \text{Recall}} \quad (6)$$

$$\text{MCC} = \frac{VP \cdot VN - FP \cdot FN}{\sqrt{(VP + FP)(VP + FN)(VN + FP)(VN + FN)}} \quad (7)$$

AUC-ROC: área sob a curva ROC (Receiver Operating Characteristic) que plota Sensibilidade vs. $(1 - \text{Especificidade})$. Invariante ao limiar de classificação. AUC = 0,5: aleatório; AUC = 1,0: perfeito.

AUC-PR: área sob a curva Precisão-Recall. Mais informativa que AUC-ROC para datasets desbalanceados (onde a classe positiva é rara).

Kappa de Cohen:

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

onde p_o é a acurácia observada e p_e é a acurácia esperada por chance.

Métrica	Quando usar	Por quê
Acurácia	Classes balanceadas	Intuitiva, mas enganosa com desbalanceamento
Precisão	FP tem alto custo	Spam filter: não perder emails legítimos
Recall	FN tem alto custo	Diagnóstico: não perder pacientes doentes
F1-Score	Equilíbrio P/R	Geral em datasets moderadamente desbalanceados
F_2	FN mais crítico	$\beta > 1$: mais peso no Recall
AUC-ROC	Threshold incerto	Ranking de probabilidades
AUC-PR	Alta desbalanceamento	Deteção de fraude, doenças raras
MCC	Desbalanceamento extremo	Métrica mais robusta para 2 classes

8.2 Estratégias de Validação

- **Holdout:** divide dados em treino/teste (e.g., 80/20). Simples, mas alta variância para datasets pequenos.
- **k-fold CV:** divide em k partes; usa $k - 1$ para treino e 1 para validação, rodando k vezes. Estimativa mais estável.
- **Stratified k-fold:** mantém a proporção de classes em cada fold. Essencial para dados desbalanceados.
- **Leave-One-Out (LOO):** caso especial com $k = n$. Quase sem viés, mas alta variância computacional.
- **Nested CV:** loop externo para avaliação, loop interno para seleção de hiperparâmetros. Evita “data leakage” de hiperparâmetros.

⚠ Overfitting vs. Underfitting

Overfitting: modelo decora os dados de treino mas não generaliza. Sintoma: erro de treino $\ll$ erro de validação. Causas: modelo muito complexo, dados insuficientes. Soluções: regularização, mais dados, redução de features, dropout.

Underfitting: modelo muito simples para capturar os padrões. Sintoma: erro de treino $\approx$ erro de validação $\gg$ erro desejado. Soluções: modelo mais complexo, mais features, menos regularização.

8.3 Testes Estatísticos de Comparação

Para comparar classificadores de forma rigorosa:

- **Teste de Wilcoxon com posto sinalizado:** compara os desempenhos de dois classificadores em múltiplos datasets. Não paramétrico; recomendado por Demšar (2006).

- **Teste de McNemar:** compara os erros de dois classificadores no mesmo dataset de teste. Usa a estatística $\chi^2 = \frac{(|e_{01}-e_{10}|-1)^2}{e_{01}+e_{10}}$ onde e_{01} é o número de exemplos classificados incorretamente pelo modelo 1 e corretamente pelo modelo 2. Dietterich (1998) recomenda este teste.
- **Teste de Friedman com post-hoc Nemenyi:** para comparar múltiplos classificadores em múltiplos datasets simultaneamente.

9 Estado da Arte (2020–2024)

XGBoost, LightGBM e CatBoost: Dominância em Dados Tabulares

Gradient Boosted Decision Trees (GBDT) dominam consistentemente benchmarks em dados tabulares estruturados, conforme revisão de Gorishniy et al. (2022) e análise de Shwartz-Ziv & Armon (2022). Principais vantagens:

- **XGBoost** (Chen & Guestrin, 2016): regularização L1/L2 integrada, computação paralela de splits via histogramas, poda de árvores por profundidade, suporte a GPU. Ganha competições Kaggle desde 2015.
- **LightGBM** (Ke et al., 2017): GOSS reduz amostras mantendo gradientes informativos; EFB agrupa features mutuamente exclusivas; crescimento leaf-wise $\Rightarrow$ 10× mais rápido que XGBoost em datasets grandes.
- **CatBoost** (Prokhorenkova et al., 2018): ordered target encoding elimina leakage em features categóricas; suporte nativo a GPU; robusto a dados heterogêneos.

Redes Neurais para Dados Tabulares

- **TabNet** (Arik & Pfister, 2021): atenção sequencial para seleção de features em cada etapa de decisão; permite interpretabilidade local por instância.
- **FT-Transformer** (Gorishniy et al., 2021): tokeniza features numéricas e categóricas e aplica Transformer; supera GBDTs em alguns benchmarks.
- **SAINT** (Somepalli et al., 2021): Self-Attention e Intersample Attention combinados; pré-treino autossupervisionado.
- **Conclusão crítica:** Gorishniy et al. (2022), após revisão sistemática, concluem que DL **ainda não supera consistentemente** GBDTs em dados tabulares. GBDTs preferidos para dados pequenos e heterogêneos; DL pode vantajar em features com alta dimensionalidade e estrutura.

 Interpretabilidade e XAI (Explainable AI)

- **SHAP** (Lundberg & Lee, 2017): valores de Shapley (teoria dos jogos cooperativos) para atribuição de contribuição de cada feature à predição. TreeSHAP é $O(TLD^2)$ para ensemble de árvores – polinomial, não exponencial.
- **LIME** (Ribeiro et al., 2016): gera perturbações locais ao redor da instância e ajusta um modelo linear interpretável para aproximar o comportamento local do classificador complexo.
- **Integrated Gradients** (Sundararajan et al., 2017): para redes neurais, integra os gradientes ao longo do caminho da baseline até a entrada.
- **Anchors** (Ribeiro et al., 2018): regras if-then de alta precisão que “ancoram” a predição local.
- **Regulação:** AI Act (EU 2024) exige explicabilidade para sistemas de IA de alto risco.

10 Exercícios

- E1. (Cálculo de Entropia e Ganho de Informação)** Dado o seguinte dataset de 10 exemplos com dois atributos binários A_1, A_2 e classes $\{+, -\}$ (6 positivos, 4 negativos): calcule $H(S)$, $IG(S, A_1)$, $IG(S, A_2)$ manualmente, identificando qual atributo ID3 escolheria como raiz. Verifique com o cálculo do Índice Gini.
- E2. (Construção de Árvore de Decisão)** Construa manualmente a árvore de decisão completa (sem poda) para o dataset “PlayTennis” clássico usando os critérios ID3 (entropia) e CART (Gini). Compare as árvores resultantes e discuta as diferenças.
- E3. (Seleção de Kernel SVM)** Para cada cenário a seguir, justifique o kernel mais adequado: (a) dados de texto com bag-of-words esparsos; (b) classificação de imagens com features extraídas por CNN; (c) dados com fronteira circular no espaço original. Implemente com scikit-learn e reporte a acurácia de validação cruzada.
- E4. (Comparação Ensemble)** No dataset “Credit Card Fraud Detection” (Kaggle), compare Random Forest, XGBoost e LightGBM em termos de AUC-PR (métrica adequada para dados altamente desbalanceados). Use stratified 5-fold CV e reporte médias e desvios-padrão. Aplique o teste de Wilcoxon para verificar se as diferenças são estatisticamente significativas.
- E5. (Seleção de Métrica para Dados Desbalanceados)** Um dataset de detecção de fraude possui 99.5% de exemplos negativos e 0.5% positivos. Um classificador que prediz sempre “não-fraude” tem acurácia de 99.5%. Calcule a Precisão, Recall, F1-Score, AUC-ROC, AUC-PR e MCC deste classificador. Discuta por que a acurácia é enganosa e qual métrica você utilizaria neste contexto.
- E6. (Tuning de XGBoost)** Implemente uma busca de hiperparâmetros (Optuna ou RandomizedSearchCV) para XGBoost no dataset Breast Cancer Wisconsin, tunando:

`n_estimators` (100–500), `max_depth` (3–8), `learning_rate` (0.01–0.3), `subsample` (0.6–1.0), `colsample_bytree` (0.6–1.0). Reporte o conjunto ótimo de parâmetros e a curva de validação de early stopping.

- E7. (Interpretação com SHAP)** Treine um XGBoost no dataset Wine Quality. Gere: (a) SHAP summary plot (beeswarm) para entender a importância global das features; (b) SHAP waterfall plot para a predição de um vinho de qualidade alta; (c) SHAP dependence plot para a feature mais importante. Discuta os insights obtidos.
- E8. (Implementação Completa)** Implemente um pipeline de classificação completo para o dataset de sua escolha incluindo: pré-processamento com `Pipeline` do scikit-learn, busca de hiperparâmetros com nested CV, comparação de pelo menos 5 algoritmos diferentes, matriz de confusão, curvas ROC e PR, e interpretação com SHAP. Escreva um relatório de 2 páginas discutindo os resultados.

11 Referências

Referências

- [1] Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81–106.
- [2] Quinlan, J. R. (1993). *C4.5: Programs for Machine Learning*. Morgan Kaufmann, San Francisco.
- [3] Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. Wadsworth, Belmont.
- [4] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- [5] Vapnik, V. N. (1995). *The Nature of Statistical Learning Theory*. Springer, New York.
- [6] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of KDD 2016*, 785–794.
- [7] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., ... & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *NeurIPS 2017*, 3149–3157.
- [8] Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: Unbiased boosting with categorical features. In *NeurIPS 2018*.
- [9] Arik, S. Ö., & Pfister, T. (2021). TabNet: Attentive interpretable tabular learning. In *AAAI 2021*.
- [10] Gorishniy, Y., Rubachev, I., Khurlov, V., & Babenko, A. (2021). Revisiting deep learning models for tabular data. In *NeurIPS 2021*.
- [11] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *NeurIPS 2017*, 4765–4774.

- [12] Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7), 1895–1923.
- [13] Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1), 119–139.
- [14] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232.